Basket Catch 5 - Arrays, Instancing, and Randomness

Randomness can be a powerful tool in programming, allowing us to have a large amount of variety with very few lines of code. Below are the two most common ways for us to create a random value in GDScript:

```
randi_range(x,y) # Generates a random int between x and y (inclusive)
randf_range(x,y) # Generates a random float between x and y (inclusive)
```

Often, we will want to store a random value for later use. We can do this by storing the output of our random methods() to a variable:

```
var my_random_float = randf_range(0, 100)
```

Data Structures: Arrays

Some times we may want to store multiple pieces of data in one place. We've said before that variables can only hold one piece of data at a time. However, this isn't entirely true. In computer science there are things called *data structures*. Data structures allow us to collect multiple pieces of data and refer to them as if they were *one* piece of data. The simplest data structure we have access to in GDScript is called an *array*. You will typically hear an *array* referred to as an *ordered collection*, let's talk more about what this means.

First, let's declare an array. Unlike a variable or function where we use a keyword to declare them we can make a new array by storing it *in* a variable. Here's an example below:

```
var my_new_array = []
```

Notice that an array is declared using square brackets. (There are other, more complex ways of declaring arrays but this is the most common way). This is what we would call an *empty* array as there is no data being stored in it. We can store as many different types of data in an array as we would like by putting them in the square brackets separated by commas in its declaration:

```
var my_new_array = [1, "six", 32, Vector2(0,0)]
```

We can then access these pieces of data by calling the correct *index*, or placement, of the data in the array. The index of an item will always be an *integer*. Arrays are indexed starting at *0*, meaning the first item in the array is in the 0-th place.

```
# 0       1        2    3  4  5  6
["hello", "world", "!", 1, 2, 3, 4]
```

To access data in an array we will use the square brackets and the *index* that we want to call:

```
my_new_array[0] # Returns the 1st item in the array
my_new_array[3] # Returns the 4th item in the array
my_new_array[2] # Returns the 3rd item in the array
```

We can also store data at an index by calling the index and setting it equal to our new data, the same way that we would assign new data to a variable:

```
my_new_array[0] = 32 # Replaces the 1st item in the array with 32
```

Creating Objects: Instancing

We will often want to create an object while our game is running. To do this we need to do something called *instancing*. This is where we create a copy of an object, an *instance*, and then add it to our scene. To get an object reference in our script we must first *preload* it. We do this by creating a variable, and using the preload() method to load a *scene* file from the filesystem into our variable. That would look like this:

```
var my_reference_object = preload("res://my_object.tscn")
```

We can then create a copy of this object by calling the instantiate() method on the variable holding the reference. We will typically want to store this *instance* in a new *local* variable as below:

```
func create_new_object():
    var my_new_object = my_reference_object.instantiate()
```

After we've created a new instance we can manipulate its properties using dot notation. Here's an example where we randomize the new object's y position:

```
func create_new_object():
    var my_new_object = my_reference_object.instantiate()
```

```
    my_new_object.position.y = randi_range(-300, 300)
```

Finally, we need to add the instance to the scene, we do this by using the get_tree().get_root().add_child() method like below:

```
func create_new_object():
    var my_new_object = my_reference_object.instantiate()
    my_new_object.position.y = randi_range(-300, 300)
    get_tree().get_root().add_child(my_new_object)
```

Here's a quick template for reference:

```
var my_reference_object = preload("res://replace_this_with_your_scene.tscn")

func create_new_object():
    var my_new_object = my_reference_object.instantiate()
    # Change the object's properties here
    get_tree().get_root().add_child(my_new_object)
```

Now it's your turn!

In your Basket Catch game do the following:

1. Create a new 2D scene and name the root node "Spawner"
2. Add a script to the root node
3. Add a variable that holds an array of all of your objects (remember, you will need to preload() each object)
4. Add a timer to the scene and connect its *timeout* signal to the root node
5. In the timeout signal function:
    1. Select a random item and create an instance of it
    2. Set the instance's x position to a random value between 10 to 600 (integers)
    3. Set the instance's y poisition to -100
    4. Set the instance's rotation to a random value between 0 and 2 * PI
    5. Set the timer's wait_time property to a random value between 1.0 and 3.0 (floats)
    6. Add the instance to the scene
6. Place the "Spawner" scene in your main scene and make sure you can continuously spawn items

After, answer the following in the "Spawner" script:

1. For the following array, give the index each value is at: ["pie", "cake", "muffin", "apple"]
    1. "cake"
    2. "apple"
    3. "pie"
    4. "muffin"
2. For the following array, give the value that would be returned for each index: [32, 54, "yes", false, true]
    1. 3
    2. 2
    3. 4
    4. 1
    5. 0